

AI Product Intelligence System Report

Approach, Design Rationale, and Results

Arik Chakraborty, Brainware University

June 22, 2026

1. Overview

This project extends a basic product-intelligence notebook into a more complete multimodal fashion system. The goal was to solve three business-oriented problems on the *Fashion Product Images (Small)* dataset:

1. Text-to-product search
2. Unique catalog creation through duplicate detection
3. Complementary product recommendation

The final system combines **CLIP** for shared text-image embeddings and **BLIP** for image captioning and explainability. This combination was chosen because CLIP is strong for retrieval and similarity tasks, while BLIP helps generate human-readable product descriptions.

2. Dataset and Preparation

The dataset contains product images and structured metadata in `styles.csv`. I first cleaned the metadata, linked each product row to its image file, removed broken image paths, and created a compact text description for every product using the following fields:

- product name
- gender
- master category
- subcategory
- article type
- base colour
- season
- usage

This text representation was important because it allowed the same product to be understood from both its image and metadata.

Data Summary

Metric	Value
Rows in styles.csv	44,424
Rows with matching images	44,419
Working subset used for indexing	3,000
Embedding dimension	512

I used a 3,000-product working subset for the main experiments to keep the Kaggle runtime practical and to make repeated testing faster. The image embeddings were cached in `/kaggle/working` so the notebook could be resumed without recomputing everything from scratch.

3. Model Choice and Why

CLIP

I used `openai/clip-vit-base-patch32` as the core retrieval model. CLIP was the best fit because it projects images and text into the same embedding space. This makes it suitable for:

- text-to-image search
- image similarity
- duplicate detection
- semantic recommendation scoring

BLIP

I used `Salesforce/blip-image-captioning-base` as a supporting model. BLIP was not the primary retrieval engine, but it added value in two ways:

- generating captions for sanity-checking image understanding
- making the overall system easier to explain in a human-readable way

This hybrid design was more useful than relying on only one model for all tasks.

4. System Pipeline

The final pipeline follows these steps:

1. Load and clean metadata from `styles.csv`
2. Link each product to a valid image
3. Build CLIP image embeddings for the indexed subset
4. Reuse the embeddings across multiple downstream tasks
5. Apply task-specific logic for search, deduplication, and recommendation

This design kept the notebook modular. Instead of creating three isolated solutions, I reused one shared representation and added targeted logic on top of it.

5. Text-to-Product Search (Task 3)

Approach

For reverse search, the system accepts a text query and converts it into a CLIP text embedding. It then computes cosine-style similarity against all product image embeddings and returns the top matches.

To improve retrieval quality, I added a **query expansion** step. For example, a query containing the word shirt is automatically expanded into related prompts such as top, casual shirt, and fashion shirt. This helps the search engine capture broader fashion semantics instead of depending on exact keyword overlap.

I also added an **explanation layer** that labels each result with reasons such as:

- colour match
- article-type match
- semantic similarity

Why This Approach

This approach matches the assignment hint about using CLIP for joint text-image understanding. It also adds creativity beyond plain nearest-neighbor search because the results are explainable and not just ranked by a raw similarity score.

Result

For the demo query “**blue casual shirt**”, the system retrieved products such as blue tops and shirts. This shows that the model understood both the colour cue and the clothing category, even when the top result was a visually similar Top rather than an exact Shirt.

The top retrieved product in the final run was:

- **Wills Lifestyle Women Blue Top**



Figure 1: Text-to-Product Search Results showing query-expanded relevant items

6. Unique Catalog Creation (Task 2)

Initial Baseline

My first duplicate-detection attempt used DBSCAN directly on the image embeddings with a cosine-distance threshold. This compressed the 3,000-product working set to **803** items and grouped **2,271** products into duplicate clusters.

Although that reduction looked impressive numerically, visual inspection revealed a problem: the clusters were too aggressive. Products with similar studio backgrounds and pose styles were being grouped together even when they were not true duplicates.

Baseline Deduplication Summary

Metric	Value
Duplicate groups found	74
Products in duplicate groups	2,271
Final catalog size	803



Figure 2: Initial Duplicate Clustering Problem (aggressive clustering mixing apparel items)

Refined Approach

To fix this, I redesigned the duplicate logic around **high-precision matching**. The refined method only considered candidate pairs that satisfied stronger constraints:

- same gender
- same master category
- same subcategory
- same article type
- very high visual similarity threshold
- same colour or unknown colour
- similar product titles, or extremely high image similarity

This refinement matters because catalog deduplication in e-commerce should prioritize precision over aggressive compression. A smaller number of high-confidence duplicate groups is more trustworthy than a large number of noisy clusters.

Refined Result

The refined approach found:

Metric	Value
Candidate image-similar pairs checked	20
Accepted duplicate pairs	7
Refined duplicate groups	7
Products in duplicate groups	14
Refined catalog size	2,993

This result is much more realistic. The refined visualization shows a near-identical black wallet pair instead of unrelated apparel items being mixed together.



Figure 3: Refined Duplicate Cluster (high-precision matching showing identical wallets)

7. Complementary Product Recommendation (Task 1)

Approach

The recommendation component was designed to suggest **complementary** items rather than visually similar ones. Since the dataset does not provide co-purchase history, I built a hybrid rule-and-embedding solution:

- rule-based compatibility by subcategory
- gender compatibility filtering
- usage compatibility filtering
- colour harmony heuristics
- CLIP-based semantic scoring from generated recommendation prompts
- a small penalty for items that are too visually similar to the source product

For example, a topwear item can recommend:

- bottomwear
- footwear
- accessories

This makes the output closer to styling recommendations than duplicate search.

Why This Approach

Task 1 could not be solved with transaction data because the dataset does not include basket history. A pure image-similarity approach would only return lookalike products, not outfit complements. The rule-guided CLIP approach was chosen to inject fashion logic into the recommendation process while still keeping the results semantic and data-driven.

Result

For the demo run, the source product was:

- **Mr.Men Boys Blazing Yellow T-shirt**

The system recommended casual shoes, a sling bag, and jeans. These recommendations were reasonable because they matched the product's casual usage and often preserved neutral or compatible colour choices.

The top recommended product in the final run was:

- **Puma Unisex Lo Punk Black Yellow Shoe**



Figure 4: Recommendation Board showing complementary items (shoes, jeans, bags) for a yellow T-shirt

8. Embedding Visualization (Additional Visuals)

To make the project more interpretable, I added a PCA-based embedding map. This reduced a sample of 1,200 image embeddings from 512 dimensions down to 2 dimensions for visualization.

The goal of this plot was not to measure final accuracy. Instead, it helps show that visually or semantically similar products tend to occupy nearby regions in embedding space.

Result

- PCA sample size: **1,200**
- PCA explained variance ratio: **0.199**

Even though PCA only preserves part of the original structure, the visualization still shows meaningful clustering patterns among subcategories such as shoes, tops, and accessories.

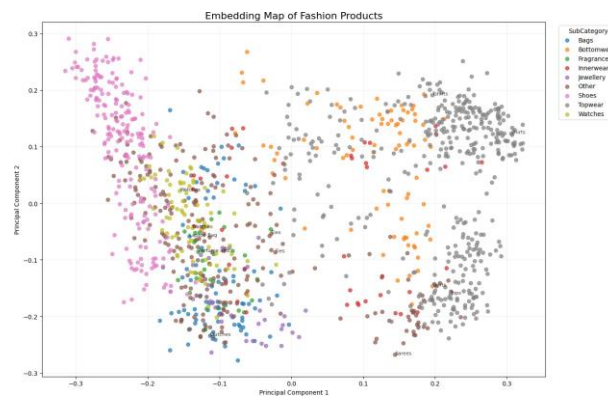


Figure 5: Embedding Map showing subcategory clusters (PCA projection of 1,200 sample products)

9. Creative Additions

Beyond the minimum assignment requirements, I added several features to improve the quality of the system:

- query expansion for stronger text retrieval
- explainable search reasons
- refined duplicate validation using metadata and title similarity

- rule-guided semantic recommendations instead of plain visual similarity
- portfolio-style result boards for search and recommendation
- embedding-space visualization for interpretability

These additions improved both the technical depth and the presentation quality of the notebook.

10. Challenges and What I Learned

The biggest challenge appeared in duplicate detection. The first image-only clustering approach looked strong numerically, but it failed qualitatively because the embeddings were influenced by common catalog photography patterns such as white backgrounds and centered subjects.

This led to an important design lesson: **good ML systems should be validated visually, not only by counts**. The refined method became better not because it found more duplicates, but because it found more trustworthy ones.

Another key lesson was that different tasks need different types of logic:

- search benefits from semantic matching
- deduplication benefits from stricter precision checks
- recommendation benefits from compatibility rules in addition to similarity

11. Final Outcome

The final notebook successfully delivers:

1. A text-to-product search engine using CLIP text-image retrieval
2. A refined duplicate-detection pipeline for cleaner product catalogs
3. A complementary recommendation engine for fashion styling
4. An interpretable embedding visualization for presentation and analysis

Overall, this project demonstrates how a single multimodal embedding backbone can support multiple e-commerce intelligence tasks when combined with task-specific reasoning and careful validation.